ETIS

Production Operations Runbook

Incident-oriented procedures for operating, diagnosing, stabilizing, and returning ETIS Engineering Studio production to service

Audience	Course owners, ETIS operators, maintainers, and incident responders
Purpose	Provide symptom-to-action runbooks for normal operations and production incidents without bypassing security, evidence, or deployment boundaries.
Production service	https://simulator.etisframework.org
Reference implementation	COMP 330/474 Software Engineering • Fall 2026
Version	Semester Start 2026 • August 23, 2026

OPERATING PRINCIPLE
Production changes must be deliberate, reversible where possible, observable, and validated. The repository and protected deployment workflow remain authoritative; local tools do not replace controlled deployment.

Contents

Executive Overview

This runbook starts from symptoms, not tools. Its purpose is to help an operator distinguish application, identity, GitHub, database, Azure runtime, and cost-control failures before making a change. The default response is inspect → classify → stabilize → recover → validate → document.

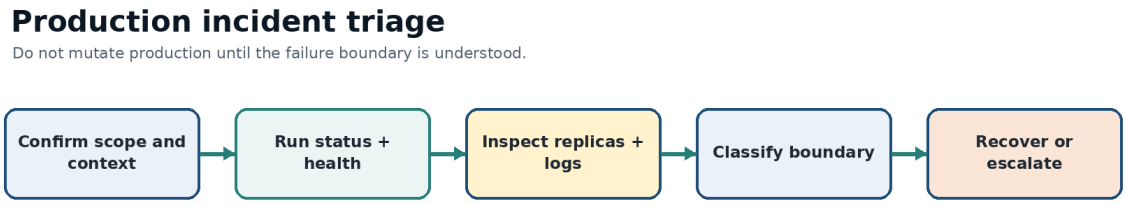

Figure 1. Production incident triage. Understand the failure boundary before changing production.

INCIDENT RULE
Do not perform speculative fixes on a healthy subsystem. Preserve the current revision, timestamps, error messages, and affected user/team context before mutation.

1. Production operating posture

Area	Normal posture
Container App	Single revision mode; min/max 1/5; at least one warm running replica during semester
Public service	HTTPS at simulator.etisframework.org
Database	Private PostgreSQL Flexible Server; migrations current
Identity	Loyola Entra authentication plus ETIS course authorization
Repository evidence	GitHub App read-only selected-repository access; GitHub OAuth identity link separate
Deployment	Protected GitHub workflow; no local deploy command
Observability	Application Insights/Log Analytics plus four core Azure Monitor alert rules
Cost	Budget $100/month; 50/80/100% actual-cost alerts; advisory telemetry

2. Monitoring and alert baseline

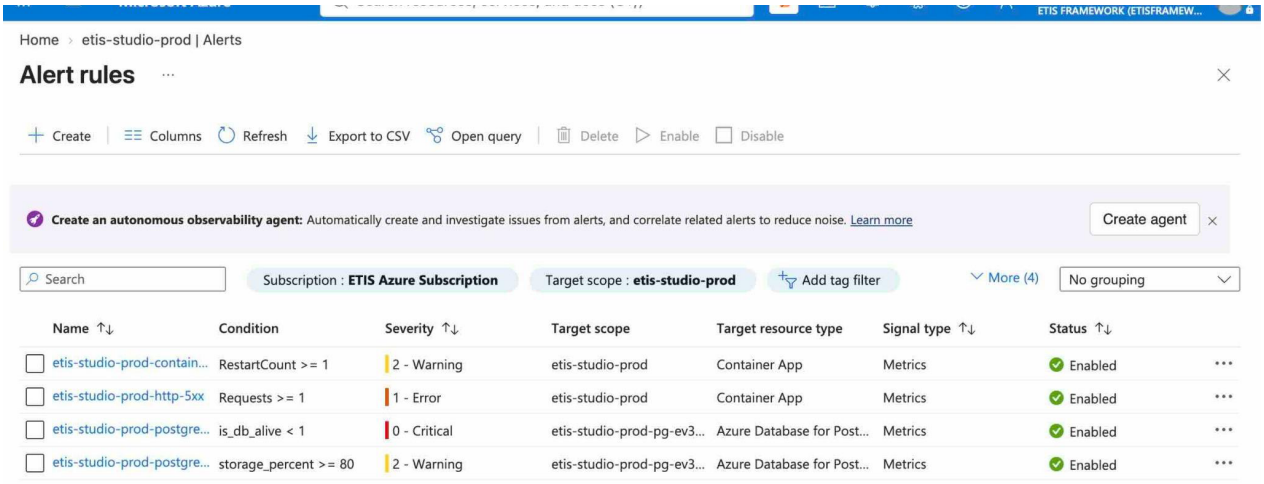

Figure 2. Production Azure Monitor alert rules. The accepted baseline covers Container App restarts, HTTP 5xx, PostgreSQL availability, and PostgreSQL storage pressure.

Alert	Condition	Severity	Response
Container restart	RestartCount >= 1	Warning	Check `replicas` and `logs`; correlate with deployments and dependency failures.
HTTP 5xx	Requests >= 1	Error	Run `health`, `logs`, and review request timing; determine whether failure is isolated or systemic.
PostgreSQL availability	is_db_alive < 1	Critical	Treat as a database availability incident. Block acceptance and move to recovery/DB diagnostics.
PostgreSQL storage	storage_percent >= 80	Warning	Review growth and capacity before exhaustion. This is a stewardship/reliability signal, not a reason to delete data.

3. Routine operating cadence

3.1 Before class / high-use period

```
./scripts/azure/etis-azure status
./scripts/azure/etis-azure health
```

- If both pass, the platform has a healthy basic operating posture.
- If a deployment occurred, run full acceptance instead.
- If students report access issues while platform health passes, shift to identity/course/GitHub triage rather than restarting Azure.

3.2 After deployment

```
./scripts/azure/etis-azure acceptance
```

The required acceptance gates are production status, health, replica capacity, external smoke test, drift, and budget controls. Cost telemetry is informational.

3.3 Weekly stewardship

- Review Azure Monitor alerts and any recurring restart/5xx pattern.
- Run `budget`; review `cost` if available.
- Review replica baseline and drift.
- Confirm recent Deploy Azure runs correspond to intended merged commits.
- Do not change capacity merely because usage increased once; look for sustained signals.

4. Incident triage procedure

1. Record incident start time, reporter, affected section/team/user, and the exact action that failed.
2. Run `etis-azure status` and `health` without changing anything.
3. If either is abnormal, run `replicas` and `logs`; capture the current/latest revision and restart/error state.
4. Classify the boundary: Azure runtime/ingress, database/migration, Entra/session, GitHub identity/App, application logic, or cost/monitoring only.
5. Use the smallest runbook below that matches the boundary.
6. After any recovery, run full production acceptance and a human spot-check of the affected user path.

CAPTURE BEFORE CHANGE
At minimum preserve: current revision, time window, failing URL/action, HTTP status or visible error, `status`/`health` output, relevant log excerpt, and whether the problem affects one user/team or everyone.

5. Runbook: Studio is unavailable

```
./scripts/azure/etis-azure status
./scripts/azure/etis-azure health
./scripts/azure/etis-azure replicas
./scripts/azure/etis-azure logs --tail 100
```

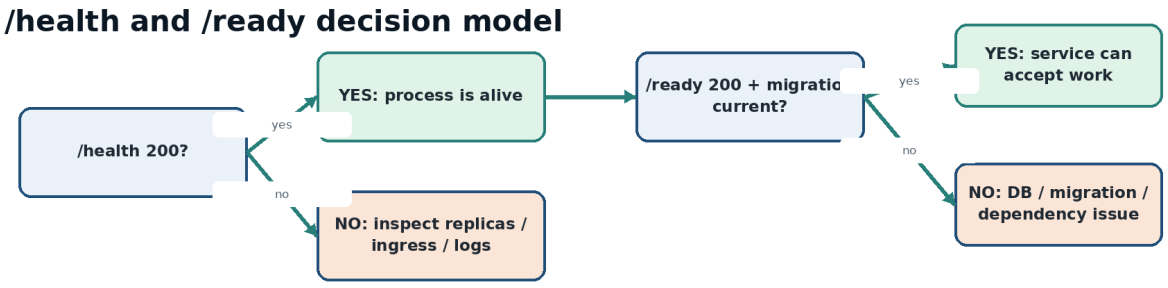

Figure 3. `/health` answers whether the process is alive; `/ready` answers whether the service is ready to perform production work.

1. If `status` cannot reach the Container App, verify Azure context and portal state.
2. If `/health` fails, prioritize replica/ingress/startup diagnostics.
3. If `/health` passes but `/ready` fails, move immediately to database/migration/dependency diagnosis.

4. If both pass but the browser fails, run `smoke` to test root/auth/bootstrap behavior.
5. Do not use `scale` to conceal a crash loop.

6. Runbook: login/authentication problem

1. Determine whether the issue is all users or one user.
2. Run `smoke`; confirm root login gate and Entra redirect behavior.
3. If platform smoke passes, verify the user is using the intended Loyola identity and is currently authorized in the course/section.
4. For one-user issues, distinguish authentication from roster/course authorization.
5. If callback/session behavior is broken for everyone, review the deployment, web origin, Entra callback alignment, and logs around `/auth/entra`/callback.
6. Do not weaken Loyola MFA or enable development login as an incident workaround.

7. Runbook: GitHub repository onboarding/access problem

1. Confirm whether GitHub identity linking is healthy separately from repository App access.
2. On My Team, identify the exact stage: no repo nominated, Step 1 owner/admin authorization, Step 2 exact verification, or previously verified repo.
3. For organization repos, GitHub organization authority may be required; do not mislabel this automatically as instructor approval.
4. Confirm GitHub App installation uses Only select repositories and includes the nominated repo.
5. Return to Studio and perform Step 2 exact verification.
6. If a verified repo must change, use the instructor bounded reset path rather than editing database state.

8. Runbook: `/ready` failure or migration mismatch

```
./scripts/azure/etis-azure health
./scripts/azure/etis-azure logs --tail 150
```

1. Treat `migration_current=False` as release-blocking.
2. Confirm PostgreSQL server availability and network/private connectivity.
3. Review the migration job/deployment workflow for the current revision.
4. Never patch the production schema manually to make readiness green.
5. If the database is unavailable or data integrity is in question, transition to the Backup/Recovery/DR Runbook.

9. Runbook: replica restart or capacity problem

```
./scripts/azure/etis-azure replicas
./scripts/azure/etis-azure logs --tail 100
./scripts/azure/etis-azure config
```

1. Check readiness, running state, restart counts, and revision.
2. Correlate restarts with deployment time and application/dependency errors.
3. Confirm min/max remains 1/5 unless an intentional override is documented.
4. Use `scale` only for an intentional temporary capacity decision after the application is healthy.
5. A request to reduce min replicas to 0 should explicitly acknowledge cold-start latency.

10. Runbook: high latency

1. Run `health` and note endpoint latency.
2. Check replicas/restarts and whether min replicas is still 1.

3. Inspect logs for slow/erroring requests and dependency timeouts.
4. Review OpenAI latency separately from core application readiness.
5. If latency is isolated to AI review turns, keep deterministic authorization/evidence paths available and communicate advisory degradation rather than declaring the whole Studio down.

11. Runbook: OpenAI degradation or outage

- AI reviewers are bounded advisers; deterministic authorization, course/term state, evidence snapshots, and existing records remain authoritative.
- Do not fabricate reviewer results or silently substitute unvalidated outputs.
- If AI calls fail, preserve the review/session state and surface the dependency failure; students can continue repository engineering work.
- Use AI Usage & Cost/telemetry and logs to distinguish latency, model/provider failure, and application errors.
- When service returns, validate a non-destructive review interaction before broad use.

12. Runbook: configuration drift or wrong revision

```
./scripts/azure/etis-azure drift
./scripts/azure/etis-azure status
./scripts/azure/etis-azure revisions
```

1. Identify the expected merged commit and successful Deploy Azure run.
2. Compare latest revision/traffic to the expected deployment.
3. For drift, decide whether the source baseline or live runtime is wrong.
4. If a bad deployment is confirmed, use controlled rollback/redeployment per the DR Runbook; do not hand-edit production until it “looks right.”

13. Runbook: cost or budget alert

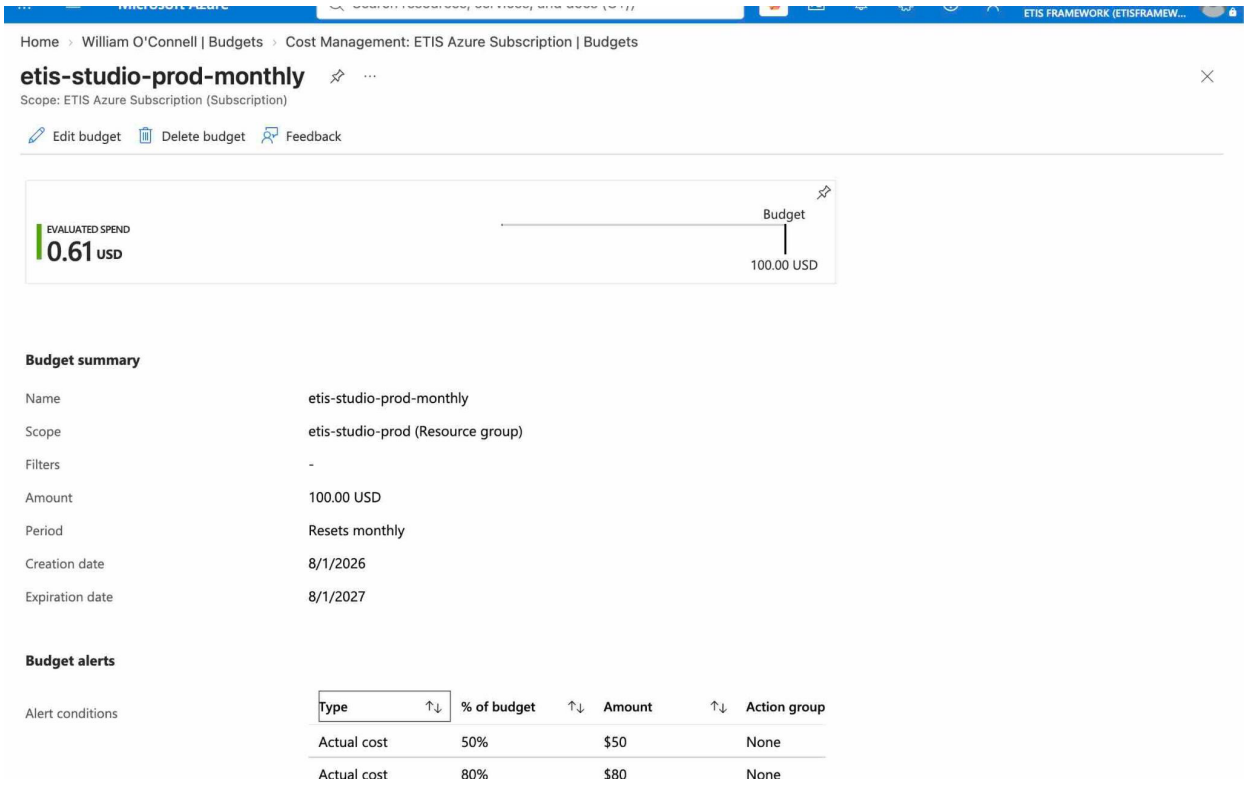

Figure 4. Production budget detail. The budget resets monthly at $100 with actual-cost alerts at 50%, 80%, and 100%.

```
./scripts/azure/etis-azure budget
./scripts/azure/etis-azure cost
```

1. Confirm the alert is actual cost and identify the threshold.
2. Review whether increased spend corresponds to expected usage, deployment churn, database/runtime growth, or an anomaly.
3. Do not automatically stop or scale down production solely because a threshold fired.
4. If `cost` is throttled by Azure, do not repeatedly retry; budget controls remain the durable signal.
5. Document any approved baseline/budget change.

14. Return-to-service

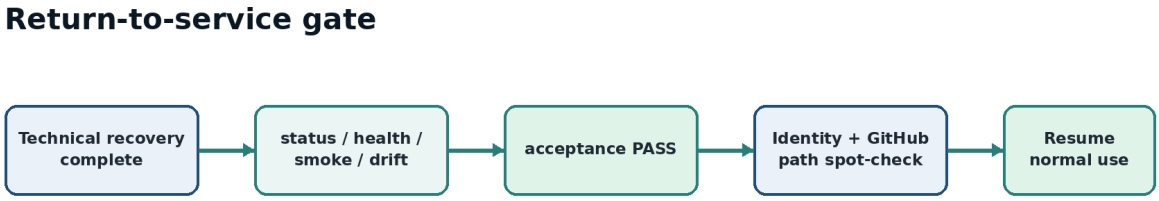

Figure 5. Technical recovery is not the end of an incident. Return to service requires full acceptance plus a human spot-check of the affected path.

1. Run `etis-azure acceptance`.
2. Confirm every required production check passes.
3. Spot-check the user path that failed (Loyola sign-in, GitHub verification, review start, etc.).
4. Confirm no temporary secret, permission expansion, or unsafe runtime override remains.
5. Record the final revision, recovery action, validation evidence, and incident end time.

15. Post-incident review

Question	Record
What happened?	User-visible symptom and technical failure boundary.
Why did it happen?	Trigger/root cause, not merely the first error observed.
What limited impact?	Fail-closed behavior, alerts, replica baseline, rollback, etc.
What changed?	Commit, configuration, secret rotation, recovery action.
How was service validated?	Acceptance output plus human path validation.
What follow-up is justified?	Specific preventive change, test, alert, documentation, or no action if healthy.

Appendix A. Incident command sequence

```
./scripts/azure/etis-azure status
./scripts/azure/etis-azure health
./scripts/azure/etis-azure replicas
./scripts/azure/etis-azure logs --tail 100
./scripts/azure/etis-azure drift
# after recovery
./scripts/azure/etis-azure acceptance
```

Appendix B. Incident record template

Field	Value to record
Start/end time	
Reporter / affected users	
Affected section/team	
Symptom/action	
Current revision	
Status/health results	
Relevant log window	
Root cause	
Recovery action	
Acceptance result	
Follow-up owner/date	